



ROAD TO HALL OF FAME

Bypass con Doble Decodificación URL ´

Actividad 10 · Reporte de laboratorio

Ingeniería en Tecnologías de la Información

Universidad Politécnica de San Luis Potosí

Fundamentos de Ciberseguridad

Mtro. Servando López Contreras

Juan Francisco Ortega Jiménez · Matrícula 184918

23 de septiembre de 2026



Road to Hall of Fame — Bypass con Doble Decodificación URL

1. Datos generales del laboratorio

Nombre del laboratorio	File path traversal, traversal sequences stripped with superfluous URL-decode
Nivel	Practitioner
Tipo de explotación	Path Traversal / Directory Traversal — bypass mediante URL-decode superfluo
Objetivo del laboratorio	Obtener el contenido del archivo /etc/passwd explotando el parámetro "filename", evadiendo un filtro que bloquea "../" pero decodifica la URL una segunda vez después de filtrar.
URL del laboratorio	https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode

2. Explicación técnica de la vulnerabilidad

Este laboratorio bloquea cualquier entrada que contenga secuencias de traversal ("../") en el parámetro filename. Sin embargo, después de aplicar ese filtro, la aplicación realiza un URL-decode adicional del valor recibido, antes de usarlo para localizar el archivo.

El servidor web ya realiza automáticamente una decodificación de URL sobre la petición entrante antes de que la aplicación inspeccione el parámetro (por ejemplo, %2f se decodifica a /). El filtro de la aplicación revisa el valor ya decodificado una vez, y si no encuentra "../", deja pasar la petición. El problema es que la aplicación, de forma redundante ("superflua"), vuelve a aplicar un URL-decode adicional sobre ese mismo valor antes de usarlo.

Esto permite construir un payload doblemente codificado: si se codifica "../" dos veces (primero a %2e%2e%2f, y luego el % de esa cadena se codifica de nuevo a %25, resultando en %252e%252e%252f), el filtro de la aplicación ve una cadena que ya no contiene "../" literal ni %2e%2e%2f (porque el servidor solo decodificó una vez, dejando %2e%2e%2f), por lo que la deja pasar. Pero al aplicar la aplicación su segunda decodificación superflua, esa cadena se

convierte en "../" real, lográndose el traversal. Esta vulnerabilidad también corresponde a OWASP Top 10 – A01:2021 Broken Access Control.

3. Procedimiento paso a paso

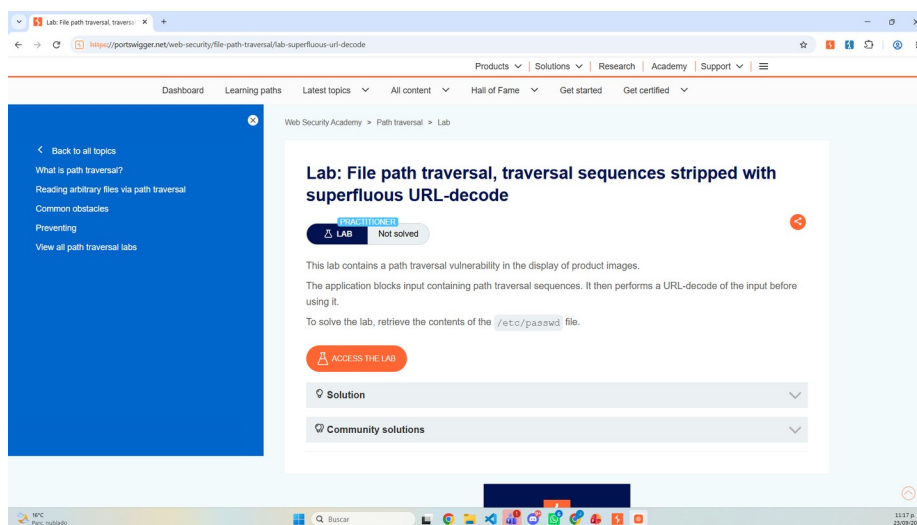
- Abrir Burp Suite Community Edition y crear un "Temporary project in memory" con la configuración por defecto.
- Acceder a la URL del laboratorio e iniciar sesión con la cuenta de PortSwigger; dar clic en "Access the lab".
- En la tienda ("We Like To Shop"), navegar entre los productos para generar tráfico hacia el servidor.
- Ajustar el filtro de HTTP history en Burp: marcar la casilla "Images" en "Filter by MIME type" y dejar vacío el campo "Hide" en "Filter by file extension", ya que el filtro por defecto oculta las peticiones de imagen.
- Con el filtro corregido, localizar en HTTP history una petición GET /image?filename=NOMBRE.jpg.
- Seleccionar esa petición, clic derecho → "Send to Repeater", e ir a la pestaña Repeater.
- En Repeater, reemplazar el valor del parámetro filename por el payload con doble codificación: `..%252f..%252f..%252fetc/passwd`, cuidando que Burp no vuelva a codificar automáticamente el símbolo % al pegarlo (debe quedar tal cual, con %25 literal).
- Enviar la petición (Send / Ctrl+Enter) y revisar el panel de respuesta: el servidor devuelve "HTTP/2 200 OK" con el contenido completo de `/etc/passwd`.
- Regresar a la pestaña del laboratorio en PortSwigger, refrescar la página y confirmar que el estatus cambia de "Not solved" a "Solved".

4. Evidencias fotográficas del proceso

A continuación se documenta, en orden cronológico, cada paso realizado durante la explotación del laboratorio, con su captura de pantalla correspondiente.

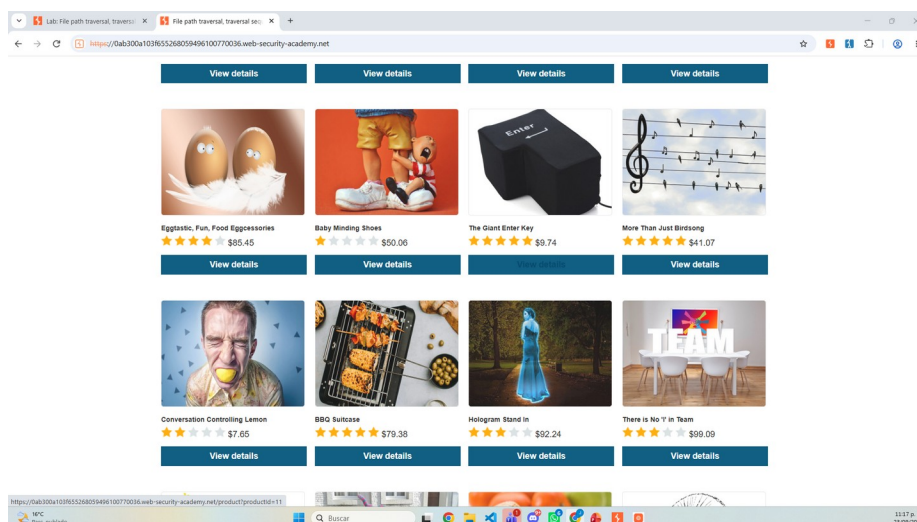
4.1 Página del laboratorio (estado inicial)

Página oficial del laboratorio en PortSwigger, mostrando el nivel "Practitioner", el estado inicial "Not solved" y la descripción del reto: la aplicación bloquea las secuencias de traversal, pero después realiza un URL-decode adicional del input antes de usarlo.



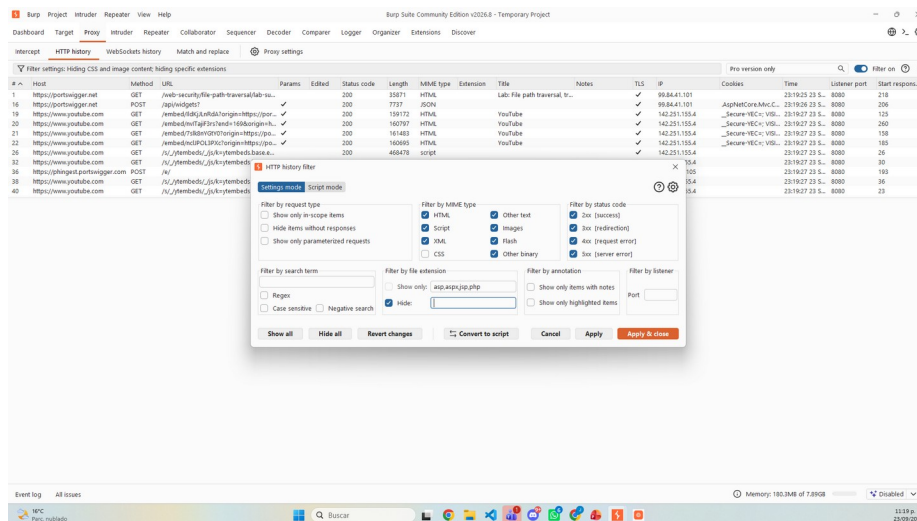
4.2 Generación de tráfico en la tienda

Se accedió a la tienda ("We Like To Shop") y se navegó entre los productos para generar peticiones GET /image?filename= hacia el servidor, necesarias para poder interceptarlas en Burp.



4.3 Ajuste del filtro de HTTP history

Se abrió el diálogo de filtros de HTTP history y se confirmó que la casilla "Images" en "Filter by MIME type" estuviera marcada, con el campo "Hide" en "Filter by file extension" vacío, para poder visualizar las peticiones a archivos de imagen.



4.4 Request interceptado

Con el filtro corregido, las peticiones GET /image?filename=XX.jpg quedaron visibles en HTTP history. Se seleccionó una de ellas (filename=60.jpg) para continuar con el ataque.

The screenshot shows the Burp Suite interface with the HTTP history tab active. A list of intercepted HTTP requests is displayed, including GET requests to various resources like /acadmin/fab/launch/0f4cd6891d7f1c, /image?filename=52.jpg, and /resources/images/vhdp.svg. The 'Send to Repeater' option is highlighted in the context menu for the selected request.

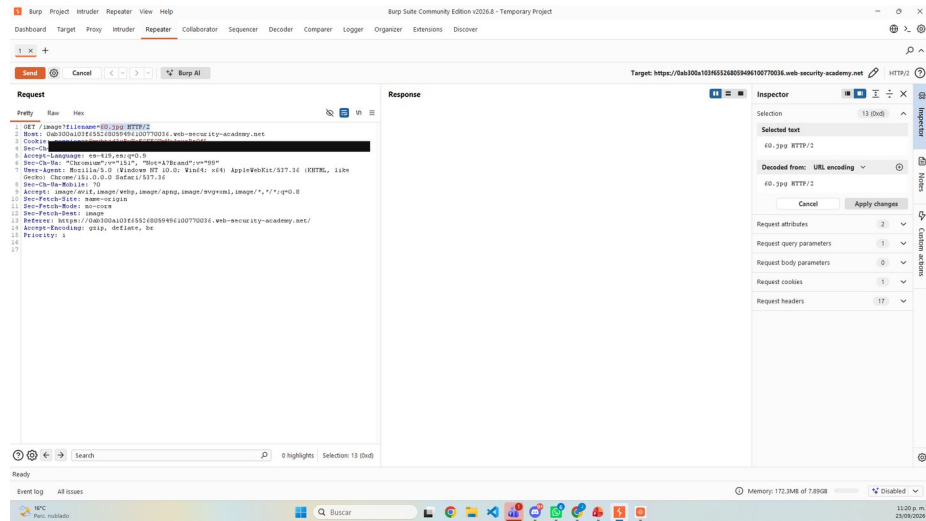
4.5 Envío de la petición a Repeater

Se hizo clic derecho sobre la petición seleccionada y se eligió la opción "Send to Repeater" del menú contextual, para poder modificar y reenviar la petición manualmente.

The screenshot shows the Burp Suite interface with the HTTP history tab active. The selected request is now highlighted in blue, and the 'Send to Repeater' option is visible in the context menu. The 'Inspector' tab is also visible, showing the request details.

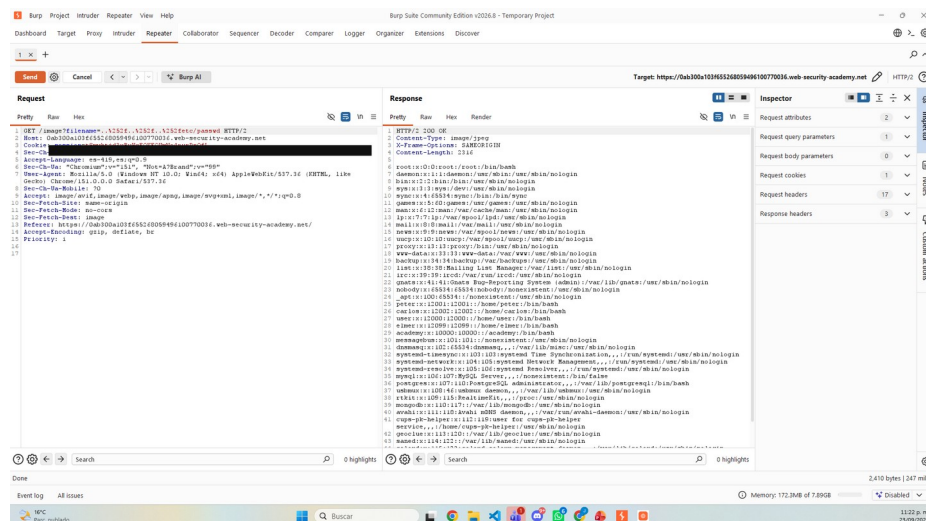
4.6 Petición cargada en Repeater

La petición GET /image?filename=60.jpg quedó cargada en Repeater, con el valor del parámetro seleccionado y listo para ser reemplazado por el payload de doble codificación.



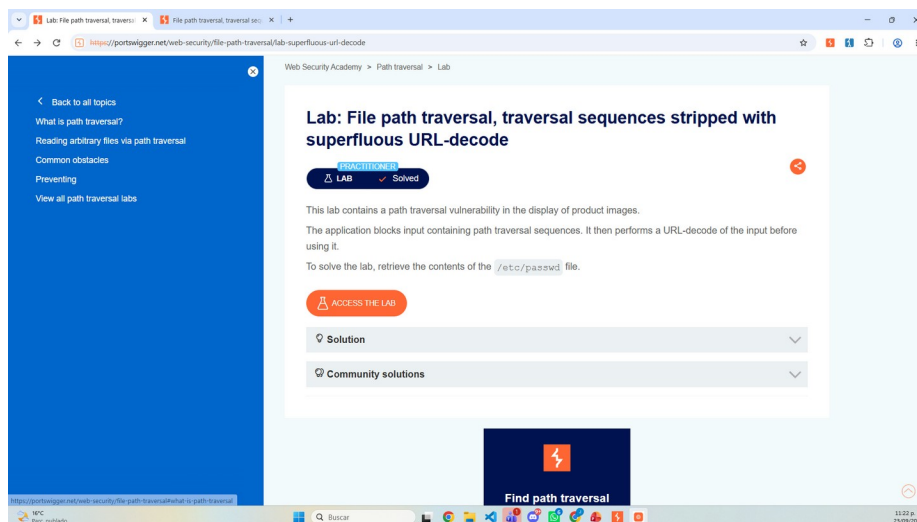
4.7 Payload de doble decodificación y respuesta vulnerable

Se reemplazó el valor del parámetro `filename` por `..%252f..%252f..%252fetc/passwd`. Al enviar la petición, el servidor respondió "HTTP/2 200 OK", devolviendo el contenido completo del archivo `/etc/passwd`, confirmando que el filtro fue evadido mediante la segunda decodificación superflua realizada por la aplicación.



4.8 Confirmación "Lab Solved"

Al regresar a la página oficial del laboratorio en PortSwigger y refrescarla, el indicador de estado cambió de "Not solved" a "Solved", confirmando que el objetivo de recuperar el contenido de `/etc/passwd` se cumplió exitosamente.



5. Explicación del payload

Payload utilizado: `..%25f..%25f..%25fetc/passwd`

- El carácter "%" se codifica en URL como "%25". Por lo tanto, la secuencia "%2f" (que representa "/" codificado) se codifica una vez más como "%252f".
- Cuando la petición llega al servidor, este realiza automáticamente una primera decodificación de URL (comportamiento estándar de cualquier servidor web), convirtiendo "%252f" en "%2f". En este punto, el filtro de la aplicación inspecciona el valor y no encuentra la secuencia "../" (ya que "%2f" aún no ha sido decodificado a "/"), por lo que la petición pasa el filtro sin ser bloqueada.
- Después de superar el filtro, la aplicación realiza una segunda decodificación de URL, de forma innecesaria ("superflua"), sobre el valor ya filtrado. Esta segunda

decodificación convierte "%2f" en "/", reconstruyendo la secuencia "../" real dentro de "../%2f" → "../".

- Al repetir el bloque "../%252f" tres veces, se generan tres secuencias "../" funcionales tras la doble decodificación, suficientes para llegar a la raíz del sistema de archivos y acceder a "etc/passwd".

6. Incidencias durante la práctica

- Filtro de contenido en Burp: al iniciar sesión de trabajo, fue necesario confirmar que el filtro de HTTP history mostrara imágenes ("Images" marcado en "Filter by MIME type"), repitiendo el ajuste realizado en actividades anteriores.
- Codificación del payload en Burp: al escribir el payload con el carácter "%" en Repeater, es importante que Burp no vuelva a codificarlo automáticamente al pegarlo o editarlo, ya que esto alteraría el doble URL-encode necesario para explotar la vulnerabilidad. El payload funcionó correctamente al escribirlo directamente como texto plano (%252f), sin usar la función de auto-codificación de Burp (Ctrl+U).

7. Mitigación recomendada

- Realizar la decodificación de la entrada una sola vez, y aplicar el filtro de secuencias de traversal después de esa única decodificación, nunca antes ni con decodificaciones adicionales posteriores al filtrado.
- Evitar decodificar el mismo valor más de una vez en distintas capas de la aplicación; cada decodificación adicional es una oportunidad para que un atacante reintroduzca caracteres previamente filtrados.

- Preferir la canonicalización de la ruta final (resolver la ruta real, por ejemplo con funciones como `realpath()`) y verificar que el resultado se encuentre dentro del directorio permitido, en lugar de depender exclusivamente del filtrado de patrones sobre el input crudo.
- Utilizar listas blancas (whitelisting) de nombres de archivo permitidos, en lugar de depender de la detección de patrones maliciosos conocidos (blacklisting).
- Aplicar el principio de mínimo privilegio, ejecutando la aplicación con permisos de sistema de archivos restringidos para limitar el impacto de una posible lectura no autorizada.

8. Conclusión

Este laboratorio permitió comprender cómo una decodificación de URL redundante, aplicada después de un filtro de seguridad, puede anular por completo la protección implementada. Se demostró de forma práctica cómo doblemente codificar un payload permite que este pase inadvertido por el filtro y se reconstruya como una secuencia de traversal funcional en una etapa posterior del procesamiento, reforzando la importancia de decodificar la entrada una única vez, antes de cualquier validación de seguridad.

Referencias

- PortSwigger. (2026). File path traversal, traversal sequences stripped with superfluous URL-decode [Laboratorio interactivo]. Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode>
- OWASP Foundation. (2021). OWASP Top 10:2021 – A01 Broken Access Control. https://owasp.org/Top10/A01_2021-Broken_Access_Control/